

**Integrate Zscaler with Identity Providers
(IdPs) such as Azure AD, Okta, and Active
Directory for seamless authentication and
authorization.**

College Name: NANDHA ARTS AND SCIENCE COLLEGE

College code: bru4j

Team ID: 69a7ffa43bc7f1a1d573957d

Team Members:

Team Leader Name: RISHIKESH P

EMAIL : Rishikesh.dp2006@gmail.com

Team member : SANJAY R

EMAIL : lingarajsandhiya38@gmail.com

Team member : SRI SANTH V

EMAIL: Vijaykumar1862006@gmail.com

Team member : POOBALAN S M

EMAIL: poobalansofficial18@gmail.com

Team member : GOWREESH M A

EMAIL: gowreeshxo74@gmail.com

Project details

Title: Integrate Zscaler with Identity Providers (IdPs) such as Azure AD, Okta, and Active Directory for seamless authentication and authorization.

This project outlines the strategy and implementation plan for integrating the Zscaler security platform with our organization's core Identity Providers (IdPs): Microsoft Entra ID (Azure AD), Okta, and on-premises Active Directory (AD). The primary objective is to create a seamless, secure, and automated framework for user authentication and authorization across all Zscaler services.

By moving away from disparate local user management and leveraging these centralized identity sources, we aim to establish a true Zero Trust architecture where access is continuously verified based on user identity and context. This initiative will consolidate authentication into a single source of truth, enforce consistent security policies like Multi-Factor Authentication (MFA), and streamline the user experience through Single Sign-On (SSO).

Ultimately, this integration will serve as the critical link between our identity infrastructure and our security enforcement point, ensuring that the right users have the right level of secure access to the internet and applications, regardless of their location.

Abstract:

This project focuses on integrating **Zscaler** with Identity Providers (IdPs) such as **Azure Active Directory (Azure AD), Okta, and Active Directory** to enable secure and seamless user authentication and authorization. As organizations move towards cloudbased services, managing user identity and access securely has become essential. Zscaler, a cloud security platform, helps protect users, applications, and data by providing secure internet access and zero-trust security.

The integration with Identity Providers allows organizations to implement **Single SignOn (SSO)** and centralized identity management. By connecting Zscaler with Azure AD, Okta, and Active Directory, users can authenticate using their existing credentials while ensuring secure access to applications and network resources. This integration also enables advanced security features such as **multi-factor authentication (MFA), rolebased access control, and policy enforcement**.

The project demonstrates how identity-based security can improve access management, reduce administrative complexity, and enhance overall cybersecurity. By implementing this integration, organizations can ensure secure, efficient, and scalable authentication while supporting modern cloud and remote work environments.

Use Case Scenarios:

1. Single Sign-On (SSO) Login

Use Case:

A user logs in through the Identity Provider (Azure AD, Okta, or Active Directory) to access multiple applications using one login.

Outputs:

1. User identity is verified.
 2. User logs in successfully with one set of credentials.
 3. Multiple applications become accessible without repeated login.
 4. Secure session is created through Zscaler.
-

2. Remote User Access

Use Case:

An employee working outside the office tries to access company resources through Zscaler.

Outputs:

1. User request is received by the system.
 2. Identity is verified through the IdP.
 3. Security policies are applied.
 4. Secure access to company resources is granted.
-

3. Multi-Factor Authentication (MFA)

Use Case:

A user tries to access a sensitive company application that requires additional verification.

Outputs:

1. User enters login credentials.
2. System requests additional verification (OTP or authentication app).
3. User completes the MFA verification.

4. Access to the secure application is granted.

4. Role-Based Access Control (RBAC)

Use Case:

Different users (admin, manager, employee) try to access system resources.

Outputs:

1. User role is identified from the Identity Provider.
2. Access policies are checked.
3. Permissions are assigned according to the role.
4. User receives allowed or restricted access.

5. User Provisioning and Deprovisioning

Use Case:

A new employee joins the organization or an employee leaves the company.

Outputs:

1. User account is created or disabled in the Identity Provider.
2. Zscaler syncs with the IdP system.
3. Access permissions are automatically updated.
4. User gets access (or access is revoked) immediately.

Stage-1

Title Of the Project: -

Replace and modernize legacy VPN architectures with a cloud-native Zscaler Zero Trust solution to reduce attack surface and lateral movement.

Overview: -

The integration of **Zscaler** with enterprise Identity Providers (IdPs) introduces a modern approach to securing access to organizational applications and resources. Earlier security models depended mainly on VPN connections and network perimeter defenses, which often provided broad access and increased the chances of internal security threats. This project utilizes Zscaler's Zero Trust Exchange platform to authenticate users at the application level instead of the traditional network level. By integrating with major IdPs such as **Microsoft Entra ID, Okta**, and on-premises **Active Directory**, organizations can create a unified identity system across their infrastructure. This integration ensures that users are authenticated in a consistent manner regardless of their device or location. It also removes the requirement for maintaining separate user directories within Zscaler and reduces manual account management tasks. Users can authenticate once and securely access authorized resources. Additionally, security administrators gain centralized control over authentication policies and user lifecycle management. The system supports modern authentication protocols such as **SAML, OpenID Connect**, and **SCIM**, enabling a strong Zero Trust security framework where identity acts as the main control point.

Integration with **Microsoft Entra ID** plays a key role for organizations that rely on Microsoft-based infrastructure. Using the SAML 2.0 protocol, a trusted federation is established between Zscaler and Azure AD to enable smooth Single Sign-On functionality. When users attempt to access Zscaler Internet Access or Zscaler Private Access portals, they are redirected to Azure AD for authentication. This allows organizations to apply Azure AD Conditional Access policies during the login process. For example, organizations can require verified devices, specific geographic locations, or risk-based authentication before granting access. In addition to authentication, the integration supports automatic user provisioning through the SCIM protocol. User accounts and group memberships are synchronized instantly from Azure AD to Zscaler, minimizing manual configuration errors and ensuring that access is immediately removed when employees leave the organization. Azure AD also provides detailed reporting and auditing features that help administrators monitor authentication activities. Overall, this integration strengthens security by combining Microsoft identity management with Zscaler's cloud security platform.

The integration with **Okta** offers a flexible and independent identity management solution for organizations implementing Zscaler. As a widely used Identity Provider, Okta connects Zscaler with numerous applications and directory services. The integration uses OpenID Connect (OIDC) to support modern and API-based authentication methods for Zscaler services. Through this approach, detailed user information such as profile attributes and group memberships can be used to enforce security policies. Okta's Universal Directory acts as the central source for user identities, which can be synchronized from HR systems and other directories. The Okta Integration Network (OIN) simplifies the deployment process by

providing ready-made connectors and setup guidelines for Zscaler. Automated provisioning through SCIM ensures that user changes in Okta are quickly reflected in Zscaler. Events such as onboarding, role modifications, or employee termination automatically update access permissions. Additionally, Okta's adaptive multi-factor authentication policies strengthen security during authentication attempts. This integration provides users with a smooth login experience while giving administrators centralized control over identity governance across Zscaler services.

Integration with on-premises **Active Directory** supports organizations that still depend on traditional infrastructure and are not fully migrated to cloud-based identity systems. Many organizations continue to maintain complex on-premises environments with structured group policies and organizational units. The Zscaler Authentication Bridge (ZAB) allows secure LDAP communication between Zscaler's cloud services and internal Active Directory servers. This lightweight connector operates behind the organization's firewall and creates outbound connections to Zscaler, removing the need for inbound firewall configurations. When users attempt authentication, requests are securely forwarded through ZAB to verify credentials against on-premises domain controllers. Group membership details are also synchronized with Zscaler to help enforce access policies and security rules. This solution enables organizations to continue using their existing Active Directory infrastructure while benefiting from cloudbased security. As a result, organizations can maintain strict access policies based on group memberships and internal structures while gradually transitioning toward modern cloud security practices.

List of Vulnerability Table:

Practice Website:

S.no	Vulnerability Name	CWE - No
1	Broken Access Control	CWE-23 Relative Path Traversal
2	Security Misconfiguration	CWE-16 Configuration
3	<i>Software Supply Chain Failures</i>	CWE-447 Use of Obsolete Function
4	Cryptographic Failures	CWE-331 Insufficient Entropy

5	Injection	CWE-114 Process Control
6	<i>Insecure Design</i>	<i>CWE- 501 Trust Boundary Violation</i>
7	<i>_Authentication Failures</i>	<i>CWE- 384 Session Fixation</i>
8	<i>Software or Data Integrity Failures</i>	<i>CWE-426 Untrusted Search Path</i>
9	<i>Security Logging and Alerting Failures</i>	<i>CWE- 778 Insufficient Logging</i>
10	<i>Mishandling of Exceptional Conditions</i>	<i>CWE – 369 Divide By Zero</i>

REPORT:

Practice Website: <http://www.itsecgames.com/>

1)Vulnerability Name: Broken Access Control

CWE-23 : Relative Path Traversal

OWASP Category: A01:2025 - Broken Access Control

Description: The product uses external input to construct a pathname that should be within a restricted directory, but it does not properly neutralize sequences such as ".." that can resolve to a location that is outside of that directory.

Business Impact: The attacker may be able to create or overwrite critical files that are used to execute code, such as programs or libraries.

Example Language: **Perl**

```
my $dataPath = "/users/cwe/profiles";
my $username = param("user");
my $profilePath = $dataPath . "/" . $username;

open(my $fh, "<", $profilePath) || ExitError("profile read error: $profilePath");
print "<ul>\n";
while (<$fh>) {
    print "<li>$_</li>\n";
}
print "</ul>\n";
```

Recommendation:

Avoid Direct User Input: The most effective way to prevent path traversal is to avoid passing user-supplied input directly to filesystem APIs.

- **Use Indirect References:** Instead of accepting a filename, use an index number or a unique ID that maps to a specific file in a server-side lookup table.
- **Implement "Allow Lists":** Only accept input that matches a strict list of permitted values or characters (e.g., alphanumeric only).
- **Apply the Principle of Least Privilege:** Ensure the application runs with the minimum necessary permissions so it cannot access sensitive system files even if a traversal occurs.

2 .Vulnerability Name: Security Misconfiguration

CWE-16: Configuration

OWASP Category: A01:2025- Security Misconfiguration

Description: It is missing appropriate security hardening across any part of the application stack or improperly configured permissions on cloud services. Unnecessary features are enabled or installed (e.g., unnecessary ports, services, pages, accounts, testing frameworks, or privileges). Default accounts and their passwords are still enabled and unchanged. A lack of central configuration for intercepting excessive error messages. Error handling reveals stack traces or other overly informative error messages to users.

Business Impact: Insecure settings can cause application crashes or allow attackers to trigger Denial of Service (DoS), preventing customers from reaching services and resulting in lost revenue.

Vulnerable Example (Node.js/Express):

javascript

```
app.use((err, req, res, next) => {  
  // Directly sending the error object reveals stack traces to the user  
  res.status(500).send(err.stack);  
});
```

Use code with caution.



Secure Configuration:

javascript

```
app.use((err, req, res, next) => {  
  console.error(err.stack); // Log internally for developers  
  res.status(500).send("An internal error occurred. Please try again later.");  
});
```

Use code with caution.



Ask anything

Recommendation:

Automate Hardening: Develop a repeatable hardening process to deploy environments that are appropriately locked down. Use automation to ensure development, QA, and production environments are configured identically.

Maintain a Minimal Platform: Remove or do not install unused features, components, frameworks, documentation, and samples to reduce the attack surface.

Enforce Least Privilege: Implement strict access controls and grant users only the minimum permissions necessary for their roles.

3.Vulnerability Name: Software Supply Chain Failures

CWE-447: Use of Obsolete Function

CWE-1021: A03:2025 - Software Supply Chain Failures

Description: A UI function for a security feature appears to be supported and gives feedback to the user that suggests that it is supported, but the underlying functionality is not implemented.

Business Impact: CWE-447 occurs when a user interface displays features that are not actually implemented or supported by the system. This can confuse users and lead to frustration when they try to use non-functional options. As a result, user trust in the application may decrease, and organizations may receive more support requests from users reporting these issues. Over time, this can reduce productivity, increase maintenance efforts, and negatively affect the company's

```
</> HTML

<!DOCTYPE html>
<html>
<head>
<title>Profile Page</title>
<script>
function deleteAccount() {
    alert("Feature coming soon!"); // Not implemented
}
</script>
</head>
<body>

<h2>User Profile</h2>
<button onclick="deleteAccount()">Delete Account</button>

</body>
</html>
```

reputation.

Recommendation:

Applications should avoid displaying features in the user interface that are not fully implemented or supported. Developers should either remove such features from the UI or clearly disable them until they are completely developed and tested. Proper validation should also be implemented to ensure that unsupported functions cannot be triggered from

the frontend or backend. Regular code reviews and UI testing should be conducted to identify incomplete features before deployment. This helps improve user experience, maintain application reliability, and prevent potential security risks.

3.Vulnerability Name: Cryptographic Failures

CWE-331: Insufficient Entropy

OWASP Category: A04:2025- Cryptographic FailuresThe product uses an algorithm or scheme that produces insufficient entropy, leaving patterns or clusters of values that are more likely to occur than others.

Business Impact: An attacker could guess the random numbers generated and could gain unauthorized access to a system if the random numbers are used for authentication and authorization.

Recommendations:

- Eliminate Predictable Seeding:

- oNever seed an RNG with easily guessable data such as the system clock, process IDs, or hardware serial numbers.Modern CSPRNG APIs typically handle seeding automatically using OS-level entropy; avoid manual seeding unless using a highentropy source.

- Increase Output Length (Bit Size):

- oEnsure the generated value has enough bits of entropy to prevent brute-force attacks.

- Avoid Hard-Coded Secret Values:

- oDo not use hard-coded salts, initialization vectors (IVs), or keys.Ensure these values are unique and randomly generated for every operation.

csharp

```
// VULNERABLE: Standard Random() is not cryptographically secure
public string GenerateOTP(int length) {
    string characters = "0123456789";
    string otp = "";
    Random rand = new Random(); // Statistical PRNG
    for (int i = 0; i < length; i++) {
        int index = rand.Next(0, characters.Length);
        otp += characters[index];
    }
    return otp;
}
```

5.Vulnerability Name: Injection

CWE- 114: Process Control

OWASP Category: A05:2025- Injection

Description: Process control vulnerabilities of the first type occur when either datum enters the application from an untrusted source and the data is used as part of a string representing a command that is executed by the application. By executing the command, the application gives an attacker a privilege or capability that the attacker would not otherwise have.

Business Impact: Exploitation can lead to unauthorized code execution that crashes services or halts production lines, resulting in significant revenue loss and missed delivery deadlines.

Specify Absolute Paths:

Always use the absolute path when loading libraries or executing external programs.

Bad: `System.loadLibrary("helper.dll");` (Searches system paths which an attacker might influence).

Good: `System.load("C:\\App\\libs\\helper.dll");` (Directly targets the intended file).

Use Hardcoded Paths:

Whenever possible, hardcode the paths to required libraries and executables within the application to prevent them from being influenced by external configuration files or registry keys.

Example Language: C

```
...
RegQueryValueEx(hkey, "APPHOME",
0, 0, (BYTE*)home, &size);
char* lib=(char*)malloc(strlen(home)+strlen(INITLIB));
if (lib) {

    strcpy(lib,home);
    strcat(lib,INITCMD);
    LoadLibrary(lib);
}
...
```

6.Vulnerability Name: Insecure Design

CWE- 501: Trust Boundary Violation

OWASP Category: A06:2025- Insecure Design

Description: A trust boundary can be thought of as line drawn through a program. On one side of the line, data is untrusted. On the other side of the line, data is assumed to be trustworthy. The purpose of validation logic is to allow data to safely cross the trust boundary - to move from untrusted to trusted. A trust boundary violation occurs when a program blurs the line between what is trusted and what is untrusted. By combining trusted and untrusted data in the same data structure, it becomes easier for programmers to mistakenly trust unvalidated data.

Business Impact:

- Unauthorized Code Execution: Attackers can exploit this to execute arbitrary commands with the privileges of the current user, potentially leading to a complete system compromise.
- Loss of Confidentiality: Sensitive information, such as administrator passwords or private encryption keys, can be fully divulged to an attacker.

Example Language: Java

```
username = request.getParameter("username");  
if (session.getAttribute(ATTR_USR) == null) {  
    session.setAttribute(ATTR_USR, username);  
}
```

Example Language: C#

```
username = request.Item("username");  
if (session.Item(ATTR_USR) == null) {  
    session.Add(ATTR_USR, username);  
}
```

Recommendations:

- Establish Clear Trust Boundaries: Explicitly define which parts of your application and data structures are "trusted" versus "untrusted".
- Perform Strict Input Validation: Always validate data before storing it in a session or using it in a trusted context.
 - o Allow-list Validation: Compare input against a list of known-good values or patterns (e.g., regex) and reject everything else.
 - o Plausibility Checks: Integrate checks at every tier (frontend to backend) to ensure the data makes sense for its intended use.

7) Vulnerability Name: Authentication Failures

CWE- 384 :Session Fixation

OWASP Category: [A07:2025 - Authentication Failures](#)

Description: Authenticating a user, or otherwise establishing a new user session, without invalidating any existing session identifier gives an attacker the opportunity to steal authenticated sessions.

Business Impact: Session Fixation occurs when an attacker forces a user to use a known session ID, allowing the attacker to hijack the session after the user logs in. This can lead to unauthorized access to user accounts and sensitive information. As a result, organizations

may experience data breaches, financial loss, and misuse of user accounts. It can also damage the company's reputation and reduce customer trust in the system.

Example Language: **HTML**

```
<form method="POST" action="j_security_check">
  <input type="text" name="j_username">
  <input type="text" name="j_password">
</form>
```

Recommendation:

To prevent session fixation attacks, applications should generate a **new session ID after a user successfully logs in** and invalidate any existing session IDs. Secure session management practices such as using **strong random session IDs, HTTPS encryption, and proper session timeout** should be implemented. Developers should also ensure that session IDs are not exposed in URLs or stored insecurely. Regular security testing and monitoring can help detect and prevent session-related vulnerabilities.

8.Vulnerability Name: Software or Data Integrity Failures

CWE- 426: Untrusted Search Path

OWASP Category: A08:2025- Software or Data Integrity Failures

Description: This might allow attackers to execute their own programs, access unauthorized data files, or modify configuration in unexpected ways. If the product uses a search path to locate critical resources such as programs, then an attacker could modify that search path to point to a malicious program, which the targeted product would then execute. The problem extends to any type of critical resource that the product trusts.

Business Impact: The program could be redirected to the wrong files, potentially triggering a crash or hang when the targeted file is too large or does not have the expected format.

Example Language: C

(bad co

```
#define DIR "/restricted/directory"

char cmd[500];
sprintf(cmd, "ls -l %480s", DIR);
/* Raise privileges to those needed for accessing DIR. */

RaisePrivileges(...);
system(cmd);
DropPrivileges(...);
...
```

Recommendations:

- Sanitize Environment Variables: Before calling external commands, clear or hard-code the PATH, LD_LIBRARY_PATH, and other search-related variables to known-safe values.

- Avoid Dangerous Search Functions: In C/C++, use functions like `execl()` or `execv()` which require explicit paths, rather than `system()`, which relies on the shell's untrusted search path.
- Secure Service Executables (Windows): Ensure all service paths are enclosed in double quotes if they contain spaces to prevent the "Unquoted Service Path" variant of this vulnerability

9)Vulnerability Name: Security Logging and Alerting Failures

CWE- 778: Insufficient Logging

OWASP Category: [A09:2025 - Security Logging and Alerting Failures](#)

Description: When a security-critical event occurs, the product either does not record the event or omits important details about the event when logging it.

Business Impact: Insufficient logging occurs when a system does not properly record important events such as user activities, errors, or security incidents. This makes it difficult for organizations to detect and investigate security attacks or system failures.

Without proper logs, identifying the source of a problem or unauthorized access becomes challenging. This can lead to data breaches, operational disruptions, financial losses, and damage to the organization's reputation.

Example Language: XML

```
<system.serviceModel>
  <behaviors>
    <serviceBehaviors>
      <behavior name="NewBehavior">
        <serviceSecurityAudit auditLogLocation="Default"
          suppressAuditFailure="false"
          serviceAuthorizationAuditLevel="None"
          messageAuthenticationAuditLevel="None" />
      ...
    </serviceBehaviors>
  </behaviors>
</system.serviceModel>
```

Recommendation:

Organizations should implement proper logging mechanisms to record important system events such as user logins, access attempts, errors, and security-related activities. Logs should be detailed, securely stored, and regularly monitored to detect suspicious behavior. Access to logs should be restricted to authorized personnel, and logs should be protected

from modification or deletion. Regular audits and automated monitoring tools should also be used to quickly identify and respond to security incidents.

10.Vulnerability Name: Mishandling of Exceptional Conditions

CWE – 369: Divide By Zero

OWASP Category: A10:2025- Mishandling of Exceptional Conditions

Descriptions: This weakness typically occurs when an unexpected value is provided to the product, or if an error occurs that is not properly detected. It frequently occurs in calculations involving physical dimensions such as size, length, width, and height.

Business Impact: This weakness typically occurs when an unexpected value is provided to the product, or if an error occurs that is not properly detected. It frequently occurs in calculations involving physical dimensions such as size, length, width, and height.

Example Language: C

```
const int DivideByZero = 10;
double divide(double x, double y){
    if ( 0 == y ){
        throw DivideByZero;
    }
    return x/y;
}
...
try{
    divide(10, 0);
}
catch( int i ){
    if(i==DivideByZero) {
        cerr<<"Divide by zero error";
    }
}
```

Recommandations:

- Input Sanitization: For user-provided data, use "allow-lists" or range checks to ensure the value falls within a safe, non-zero interval.
- Check Collection Sizes: When calculating averages or percentages based on the count of items in a list, verify the collection is not empty.

Stage 2

Overview :-

Nessus is a vulnerability scanning tool created by Tenable, Inc.. It functions as a remote scanner, which means it can analyze systems without needing to be installed directly on the target machine. The tool works by sending multiple types of probes to the target system to detect potential security weaknesses. These probes may include network packets, operating system commands, and requests sent to applications.

Nessus operates using a plugin-based framework. Plugins are small programs designed to detect particular security vulnerabilities. More than 1,200 plugins are available in Nessus, enabling it to identify a wide range of issues across operating systems, software applications, and network devices.

When a scan begins, Nessus first uses discovery plugins to determine the operating system and the services running on the target host. After identifying this information, it executes multiple plugins that test the system for possible vulnerabilities. For every vulnerability detected, Nessus generates a detailed report that includes the severity level, instructions to reproduce the issue, and recommended steps to fix or mitigate the problem.

Nessus can scan different types of targets, such as individual computers, entire networks, and web applications. It also allows users to scan specific ports and services when required. Scans can be performed manually or scheduled to run automatically at specific times.

Working Process of Nessus

The following steps explain how Nessus performs a vulnerability scan:

Nessus establishes a connection with the target system using protocols such as TCP, UDP, and ICMP.

It sends multiple probes to identify the operating system and active services running on the host.

The scanner then executes various plugins to check for known vulnerabilities.

After completing the scan, Nessus generates a report that outlines the detected vulnerabilities, their severity levels, reproduction steps, and possible remediation methods.

Nessus is widely used by cybersecurity professionals to detect and fix vulnerabilities within networks. Its ease of use and detailed reporting features help organizations prioritize and address security issues effectively.

Advantages of Using Nessus

Wide vulnerability detection: With more than 1,200 plugins, Nessus can identify numerous vulnerabilities across operating systems, applications, and network devices.

User-friendly interface: The tool provides an intuitive interface that simplifies the configuration and execution of scans.

Detailed reporting: Nessus produces comprehensive reports describing vulnerability severity, reproduction procedures, and remediation steps.

Scalability: It can scan various targets including individual hosts, full networks, and web applications.

Flexible scanning: Users can run scans manually or schedule them to run automatically.

Overall, Nessus is a powerful security scanning solution that helps organizations strengthen their network security. It serves as an important tool for cybersecurity professionals aiming to identify and resolve system vulnerabilities.

S.no	Vulnerability name	Severity	plugins
1	SQL Injection	Critical	11219
2	Cross-Site Scripting	High	10815
3	Directory Traversal	High	10547
4	Cross-Site Request Forgery	Medium	10287
5	Information Disclosure	Medium	10107
6	Improper Input Validation	High	10335
7	Broken Authentication	Medium	10690
8	Unrestricted File Upload	High	10933

REPORT: -

1,Vulnerability Name: -Information Disclosure

Severity: Medium *Plugin:* **11219**

Port: 80/tcp

Description

The web application fails to properly validate user inputs before using them in SQL queries. Attackers can manipulate input parameters to execute malicious SQL commands and access or modify the backend database.

Solution

- Use **prepared statements and parameterized queries**
- Validate and sanitize user input
- Implement a **Web Application Firewall (WAF)**
- Restrict database privileges

Business Impact

Attackers could gain unauthorized access to sensitive information such as user credentials and financial records, leading to **data breaches and loss of customer trust**.

2,Vulnerability Name: Cross-Site Scripting

Severity: High

Plugin: **10815**

Port: 80/tcp

Description

The application does not properly sanitize user input before displaying it in the browser. Attackers can inject malicious JavaScript that executes in users' browsers.

Solution

- Implement **input validation**
- Encode user input before displaying it
- Use **Content Security Policy (CSP)** headers

Business Impact

Attackers could steal session cookies, hijack user sessions, and redirect users to malicious websites.

3,Vulnerability Name: Directory Traversal

Severity: High

Plugin: **10547**

Port: 80/tcp

Description

The web application improperly validates file path inputs, allowing attackers to access files outside the intended directory using path traversal sequences like ../.

Solution

- Restrict file path inputs
- Implement **input validation**
- Use secure file access APIs

Business Impact

Attackers may access sensitive system files such as configuration files or password files.

4,Vulnerability Name: Cross-Site Request Forgery

Severity: Medium

Plugin: **10287**

Port: 80/tcp

Description

The application does not verify whether a request was intentionally made by the authenticated user, allowing attackers to perform unauthorized actions on behalf of victims.

Solution

- Implement **CSRF tokens**
- Validate HTTP referrer headers
- Require reauthentication for sensitive actions

Business Impact

Attackers may perform unauthorized actions such as modifying account details or performing transactions.

5,Vulnerability Name: Information Disclosure

Severity: Medium

Plugin: **10107**

Port: 80/tcp

Description

The web server exposes sensitive information such as server version, software framework details, or error messages that reveal internal system information.

Solution

- Disable verbose error messages
- Hide server version details
- Configure secure server headers

Business Impact

Attackers can use this information to identify vulnerable software versions and launch targeted attacks.

6,Vulnerability Name: Improper Input Validation

Severity: High

Plugin: **10335**

Port: 80/tcp

Description

The application does not properly validate user inputs before processing them. This may allow malicious data to be processed by the system.

Solution

- Validate all input fields
- Use strict input validation rules
- Implement server-side validation

Business Impact

Improper validation may lead to other vulnerabilities such as SQL injection or XSS, which could compromise system security.

7,Vulnerability Name: Broken Authentication

Severity: Medium

Plugin: **10690**

Port: 80/tcp

Description

The authentication system may contain weaknesses such as predictable session IDs or insufficient authentication checks, allowing attackers to bypass login mechanisms.

Solution

- Implement strong authentication mechanisms

- Use secure session management
- Enforce multi-factor authentication (MFA)

Business Impact

Attackers could gain unauthorized access to user accounts and sensitive information.

8,Vulnerability Name: Unrestricted File Upload

Severity: High

Plugin: **10933**

Port: 80/tcp

Description

The application allows users to upload files without properly validating the file type or content. Attackers could upload malicious scripts that execute on the server.

Solution

- Restrict allowed file types
- Validate file content and MIME types
- Store uploaded files outside the web root
- Rename uploaded files

Business Impact

Attackers may upload malicious files that allow **remote command execution**, potentially leading to full system compromise.

Stage-3

Title: - Integrate Zscaler with Identity Providers (IdPs) such as Azure AD, Okta, and Active Directory for seamless authentication and authorization.

- *Legacy Authentication Challenges*

Before implementing **Zscaler–IdP integration**, organizations often encounter several difficulties with traditional authentication systems. Conventional **Virtual Private Network** architectures usually follow an “authenticate once, trust indefinitely” approach, where users are given broad network access after a single successful login. This model can create serious security risks because users may receive more permissions than necessary, allowing potential attackers to move across systems within the network. VPN gateways also need to be accessible from the public internet, exposing them to threats such as port scanning, brute-force attacks, and vulnerability exploitation. In addition, remote user traffic is often routed through the organization’s data center before reaching the internet, which increases latency and reduces performance. Hardware-based VPN devices also have limited capacity and require manual upgrades during times of heavy usage. IT teams must regularly update, patch, and maintain these physical appliances. Another major challenge is the lack of integration with modern identity providers like **Microsoft Entra ID** or **Okta**, which forces administrators to manage multiple user accounts across different systems. These issues highlight the need for a modern identity-focused approach where authentication is handled by enterprise IdPs and access is granted based on verified user identity rather than network location.

- *Zero Trust Identity Principles*

The integration of **Zscaler** with Identity Providers is based on the principles of **Zero Trust Architecture**, which changes the traditional approach to authentication and authorization. The key idea of “never trust, always verify” requires every access request to be properly authenticated and authorized, regardless of whether the user is inside the office or working remotely. The concept of “least privilege access” ensures that users receive only the minimum permissions required to perform their tasks, which are controlled through identity-based policies instead of network location. Another important principle is “assume breach,” where systems are designed with the expectation that attackers may already be present, making continuous verification necessary for every session. “Micro-segmentation” further strengthens security by separating users and applications, preventing attackers from moving across systems within the network. In this model, identity becomes the main security boundary, replacing the older approach where the internal network was automatically trusted.

Authentication must consider multiple factors such as user identity, device condition, location, and behavior. Access permissions are then determined dynamically based on real-time risk evaluation instead of fixed rules. These concepts guide the integration of Zscaler

with identity providers, creating a security environment where identity controls access and users connect only to specific applications rather than the entire network.

- *Supported Identity Providers and Protocols*

Identity Provider Integration and Protocol Support

Zscaler can integrate with a wide variety of identity providers using standard authentication protocols, allowing organizations to adapt the solution to different enterprise environments. Integration with Microsoft Entra ID (Azure AD) uses **SAML 2.0** for federated authentication and **SCIM** for automated user provisioning, enabling organizations to apply Conditional Access policies and use Azure AD's reporting tools. Integration with Okta uses **OpenID Connect (OIDC)** for modern authentication and SCIM for managing the user lifecycle, with ready-made connectors available through the Okta Integration Network (OIN) to simplify configuration. For organizations using onpremises infrastructure, integration with Active Directory is performed through the Zscaler Authentication Bridge (ZAB). This lightweight connector allows secure LDAP communication between Zscaler cloud services and internal domain controllers without opening inbound firewall ports. The ZAB operates behind the firewall, establishing outbound connections to Zscaler while forwarding authentication requests and synchronizing group membership information. Zscaler also supports other identity providers such as Ping Identity, OneLogin, and standard SAML providers. Supported protocols include SAML 2.0 for browser-based single sign-on, OIDC for modern applications and mobile access, LDAP for directory integration, and SCIM for automated user provisioning, ensuring compatibility with different access scenarios and devices.

Prerequisites and Licensing Requirements

To successfully implement Zscaler-IdP integration, certain prerequisites and licenses must be available beforehand. Organizations need active subscriptions for **Zscaler Internet Access (ZIA)** and/or **Zscaler Private Access (ZPA)** with sufficient user licenses that include support for SSO and SCIM provisioning. Administrators must have appropriate access to the Zscaler Admin Portal to configure authentication and identity provider settings. For cloud identity providers like Microsoft Entra ID or Okta, administrative privileges are required to create enterprise applications, configure SAML settings, and manage API permissions for SCIM provisioning. In environments using onpremises Active Directory, a Windows Server system is required to host the Zscaler Authentication Bridge with internet connectivity to communicate with Zscaler cloud services. Network requirements include firewall rules that allow outbound HTTPS (TCP 443) traffic from ZAB servers to Zscaler endpoints and from user devices to Zscaler enforcement nodes. Proper DNS configuration is also necessary for both Zscaler services and internal domain controllers. In addition, organizations must manage SSL certificates for SAML assertion signing and determine which directory attributes—such as email addresses, user principal names, and group memberships—will be used for policy enforcement and application access.

Discovery and Requirements Gathering

The discovery phase is essential for preparing a successful integration by analyzing the current identity environment and access needs. This process begins with identifying all internal applications currently accessed through legacy VPN systems, such as student information systems, ERP solutions, library databases, research computing environments, internal file servers, and administrative portals. For each application, important technical details must be recorded, including protocols (such as HTTP/HTTPS, RDP, SSH, or SMB), port numbers, hostnames, IP addresses, and the user groups that require access. Organizations should also conduct user segmentation discussions to classify users based on their roles and access requirements—for example, faculty members accessing research servers, students using library resources, staff working with administrative systems, IT teams managing infrastructure, and external collaborators with limited access. In addition, existing identity sources should be reviewed to determine which directory systems—such as Azure AD, Okta, or on-premises Active Directory—serve as the primary identity repositories for different user groups. Current authentication methods should also be documented, including multi-factor authentication requirements, password policies, and any existing single sign-on implementations.

Identity Source Architecture Planning

Planning the identity source architecture involves deciding which directories will act as the main sources of identity information for different groups of users. In organizations with hybrid identity environments, it is important to determine whether Microsoft Entra ID will function as the primary authentication system with synchronization from on-premises Active Directory through Azure AD Connect, or whether direct on-premises integration using the Zscaler Authentication Bridge is more suitable for certain scenarios. The synchronization structure between directories should be clearly documented, including which user attributes are synchronized, how group memberships are handled, and how frequently updates occur. Attribute mapping must also be planned between identity provider directories and Zscaler user attributes to ensure fields such as `userPrincipalName`, email, given name, surname, and group membership are correctly aligned. When integrating with Azure AD, organizations must decide whether to use SAML-based authentication with optional SCIM provisioning or adopt OpenID Connect for modern authentication needs. For Okta integration, administrators should plan the Universal Directory structure and identify which user profile attributes will be shared with Zscaler during authentication and provisioning. In environments that rely on on-premises Active Directory through ZAB, it is also necessary to document domain controller locations, network latency between sites, and requirements for high availability of the connector. Disaster recovery strategies should be considered as well, including identity provider failover and offline access options. Finally, organizations should design a phased migration strategy that begins with pilot users before expanding to the entire organization.

Application Discovery and Categorization

A detailed application discovery process ensures that all resources needing secure access are identified before policies are configured. This can be done by using network monitoring tools or reviewing firewall logs to determine which internal applications and services remote users have accessed during the previous months. For every identified application, technical details such as the fully qualified domain name or IP address, communication protocol, port number, and application type should be recorded. Discussions with application owners and department managers can help confirm the accuracy of discovered systems and reveal any unofficial or shadow IT applications that may not appear in logs. Applications should then be grouped according to sensitivity levels, for example public, internal, confidential, or restricted, so that appropriate security policies can be applied. It is also important to record application dependencies, such as backend databases or authentication services, to ensure that segmentation does not disrupt functionality. Some applications may share infrastructure but require different access policies, which should also be documented. Maintaining a structured inventory of applications—including details like application name, description, department owner, protocol, port number, destination address, user groups, and sensitivity level—helps administrators manage and validate the discovery results. These findings should be reviewed with IT teams to ensure that no critical systems are overlooked. Applications can then be prioritized for initial configuration in Zscaler Private Access based on their importance to business operations and their reliance on VPN access.

User Group Definition and Attribute Planning

Establishing user groups and planning directory attributes is essential for implementing detailed access control policies in Zscaler. Organizations should collaborate with HR departments and management teams to identify different roles that require specific access permissions, such as faculty members, staff, students, researchers, IT administrators, and external partners. The relationship between organizational roles and directory groups must be documented, including whether the groups originate from on-premises Active Directory, are managed within Microsoft Entra ID, or are maintained in Okta. Group nesting strategies can also be used to simplify policy management—for example, creating a general faculty group that includes smaller departmental groups within it. Important user attributes needed for policy decisions should also be identified, such as department, job title, location, employee type, reporting manager, and contract duration for temporary employees. Multi-valued attributes, especially group memberships, should be carefully documented because they can be used to automatically apply dynamic access policies within Zscaler.

Infrastructure Readiness Assessment

Evaluating infrastructure readiness ensures that the environment can effectively support the integration of **Zscaler** with identity providers without causing performance or reliability

problems. Organizations should review network bandwidth and latency between user locations and nearby Zscaler data centers to ensure optimal connectivity. Firewall and proxy configurations must allow outbound HTTPS traffic to Zscaler cloud domains while avoiding SSL inspection that might disrupt authentication. DNS systems should be checked to guarantee proper resolution of both Zscaler service domains and internal application addresses. Infrastructure capacity must also be assessed to confirm that sufficient resources such as CPU, memory, and storage are available for deploying Zscaler App Connectors. High availability planning should include deploying multiple connectors across different failure domains. Additionally, the capacity of identity provider infrastructure—such as authentication throughput limits or directory controller performance—should be reviewed. Endpoint readiness must also be confirmed by checking whether user devices and operating systems are compatible with the Zscaler Client Connector. Finally, connectivity tests should be performed from different user locations, and any network devices that might block Zscaler traffic should be identified and configured with the necessary exceptions.

Azure AD Integration – SAML Configuration

Setting up SAML federation between **Microsoft Entra ID** and Zscaler creates a trusted authentication relationship. Administrators begin by creating a new enterprise application in the Azure AD portal and selecting Zscaler from the application gallery. After the application is created, the Single Sign-On method is configured using SAML. The administrator must enter the Identifier and Reply URL values provided by the Zscaler Admin Portal to establish the authentication connection. User attributes and claims are then configured so that Azure AD attributes are passed to Zscaler during authentication. Important fields such as user principal name, email address, first name, surname, and group memberships should be mapped correctly. Group claims are also configured so that security groups can be used for policy enforcement. After completing the configuration, the federation metadata file is downloaded from Azure AD and uploaded to Zscaler to finalize the trust relationship. Testing the SAML configuration ensures the authentication process works correctly before moving forward.

Azure AD Integration – SCIM Provisioning Setup

SCIM provisioning between Azure AD and Zscaler enables automatic management of user accounts. Once provisioning is enabled, user accounts are automatically created, updated, or removed based on directory changes. Administrators configure the SCIM endpoint and authentication token provided by the Zscaler Admin Portal within the Azure AD provisioning settings. After verifying the connection, attribute mappings must be reviewed to ensure that Azure AD user attributes correctly correspond to Zscaler's user schema. Group synchronization can also be configured so that Azure AD groups are automatically mapped to Zscaler groups used for access policies. To safely deploy the integration, organizations typically begin with a limited scope by synchronizing only selected users and groups. Provisioning logs should be monitored to identify potential errors, and test users should be validated within the Zscaler Admin Portal before expanding synchronization to additional users.

Okta Integration – OIDC Application Configuration

Configuring **Okta** as an identity provider for Zscaler using OpenID Connect enables modern authentication for cloud services. Administrators create a new application integration in the Okta Admin Console and select OpenID Connect as the authentication method. The Zscaler callback URL is added as the sign-in redirect URI, and optional logout URLs can be configured for single logout functionality. Access to the application is controlled by assigning it to selected users or groups during the pilot phase. Required API scopes such as profile, email, groups, and offline access must be granted to allow Zscaler to retrieve identity information. The Client ID and Client Secret generated by Okta are then used during Zscaler configuration. Optional security policies such as multi-factor authentication can also be enforced through Okta. After configuration, the authentication process should be tested to confirm that user attributes and group information are successfully transmitted to Zscaler.

Okta Integration – SCIM Provisioning Configuration

SCIM provisioning can also be configured between Okta and Zscaler to automate user and group synchronization. Administrators enable API integration in the Okta provisioning settings and enter the SCIM endpoint and API token obtained from the Zscaler Admin Portal. After confirming connectivity, provisioning settings are configured to allow automatic user creation, profile updates, and account deactivation. Attribute mappings should be reviewed to ensure that Okta profile attributes align with the Zscaler user schema. Group synchronization can also be enabled so that selected Okta groups are automatically pushed to Zscaler for access policy management. Administrators should test provisioning by creating and assigning a sample user, verifying that the account appears in Zscaler. Removing the assignment should also deactivate the user in Zscaler to confirm correct deprovisioning behavior.

On-Premises Active Directory Integration – ZAB Deployment

The Zscaler Authentication Bridge (ZAB) allows secure communication between Zscaler cloud services and on-premises **Active Directory** environments. To deploy ZAB, administrators install the connector on Windows Server systems that are connected to the domain and have internet access. The connector operates behind the firewall and establishes outbound connections to Zscaler, removing the need for inbound firewall rules. After installation, the connector authenticates with Zscaler using a token generated in the Zscaler Admin Portal. The service must also have appropriate permissions to query domain controllers through LDAP. Once configured, administrators verify connectivity by checking service logs and confirming that the connector appears as active in the Zscaler portal. Additional connectors can be deployed to provide redundancy and load balancing.

On-Premises Active Directory Integration – Directory Connector Configuration

After deploying ZAB, administrators configure directory synchronization settings within the Zscaler Admin Portal. A directory connector is created and configured with the appropriate

domain controller addresses and LDAP ports. For secure communication, LDAPS can be enabled if domain controller certificates are trusted by the connector servers. Administrators also specify search base locations and filters to ensure that only relevant user accounts and groups are synchronized. Attribute mapping ensures that Active Directory fields such as `userPrincipalName`, email, and group memberships are properly mapped to Zscaler user attributes. Synchronization schedules can be configured for full and incremental updates. Testing the connection confirms that users and groups are successfully imported into Zscaler. Logging and monitoring should be enabled during the initial deployment phase to detect synchronization issues.

Conclusion

In summary, this project explored the application of security technologies and modern identity frameworks to strengthen web security and access control. Early stages focused on understanding web application testing and vulnerability assessment, highlighting the importance of identifying and mitigating potential risks in web systems. Tools such as **Nessus** provided valuable insights into security weaknesses and supported the development of stronger security strategies.

The final stage demonstrated how replacing traditional VPN-based architectures with a cloud-based Zero Trust solution significantly improves organizational security. By implementing **Zscaler** services and integrating them with enterprise identity providers, organizations can shift from network-based access control to an identity-driven security model. With **Zscaler Private Access**, users are connected only to specific applications rather than the entire network, reducing the attack surface and preventing lateral movement. This approach enhances security, improves performance for remote users, and creates a scalable foundation for modern hybrid work environments.

The integration also enables Single Sign-On, automated user provisioning, and consistent enforcement of security policies such as multi-factor authentication. By leveraging standard protocols including SAML, OpenID Connect, and LDAP, the system integrates smoothly with existing identity infrastructure. The result is a secure, efficient, and future-ready authentication framework that supports the principles of Zero Trust security.

Tools explored: -

Nmap, Metasploit, Nessus, Kali Linux, OWASP ZAP, Burp Suite.

